

Azure Data Factory End-to-End Data Pipeline Using Azure Blob Storage

Project Overview

The purpose of this project was to gain practical experience with Microsoft Azure cloud services by designing and implementing an end-to-end data pipeline using Azure Storage Account and Azure Data Factory (ADF). The project focused on understanding how data can be stored in Azure Blob Storage, accessed through Azure Data Factory, validated using metadata activities, transferred through a pipeline, and monitored through Azure's cloud-based monitoring tools.

The dataset used for this project was the Superstore Dataset, which was uploaded as a CSV file into Azure Blob Storage. The entire workflow was built and executed within the Azure environment, providing hands-on experience with cloud-based data integration and orchestration.

Resource Group Creation

The first step involved creating a Resource Group named RG_DataPipeline_Demo in the Central India region. A Resource Group acts as a logical container that organizes and manages related Azure resources. Creating a dedicated Resource Group simplified resource management and ensured that all project components remained grouped together within a single environment.

This Resource Group served as the foundation for deploying and managing the Storage Account and Azure Data Factory resources required for the project.

Storage Account and Blob Storage Configuration

After creating the Resource Group, an Azure Storage Account named datapipelinstorage01 was provisioned. The storage account was configured using the following settings:

- Subscription: Azure for Students
- Resource Group: RG_DataPipeline_Demo
- Region: Central India
- Performance Tier: Standard
- Redundancy Type: Locally Redundant Storage (LRS)

The Storage Account provides scalable and secure cloud storage services. Within the Storage Account, a Blob Container named sales-data was created. Blob Storage is commonly used for storing unstructured data such as CSV files, images, videos, and documents.

The file sales_data.csv was uploaded successfully into the container. This dataset acted as the source data for the pipeline implementation.

Azure Data Factory Creation

The next phase involved creating an Azure Data Factory instance named ADF_DataPipeline_Demo. Azure Data Factory is a cloud-based data integration service used for creating, scheduling, and monitoring data workflows.

After deployment, the ADF Studio interface was explored to understand its primary modules:

Author

The Author section is used for building pipelines, creating datasets, and configuring activities.

Monitor

The Monitor section provides execution details, run history, success and failure status, and performance metrics for pipeline runs.

Manage

The Manage section is used for configuring Linked Services, Integration Runtimes, triggers, and access controls.

Understanding these components was essential before proceeding with pipeline development.

Linked Service Configuration

To establish communication between Azure Data Factory and Azure Blob Storage, a Linked Service named BlobStorageConnection was created.

A Linked Service acts as a connection string that stores authentication and connectivity information for external data sources.

The Linked Service was configured using:

- Azure Blob Storage Connector
- Storage Account: datapipelinstorage01
- Integration Runtime: AutoResolveIntegrationRuntime

A connection test was performed successfully, confirming that Azure Data Factory could access the Storage Account without any issues.

Dataset Creation

Datasets represent the structure and location of data within Azure Data Factory.

Two datasets were created:

Source Dataset

The source dataset was configured using:

- File Type: Delimited Text
- File Name: sales_data.csv
- Container Name: sales-data
- Encoding: UTF-8
- Delimiter: Comma (,)
- First Row as Header: Enabled

This dataset pointed to the uploaded CSV file stored in Blob Storage.

Destination Dataset

A destination dataset was also created to serve as the target location for copied data during pipeline execution.

Creating both datasets enabled Azure Data Factory to understand the source and target data structures required for the pipeline.

Metadata Validation Using Get Metadata Activity

Before transferring data, metadata validation was performed using the Get Metadata activity.

A pipeline named MetadataPipeline was created. The Get Metadata activity was configured to access the source dataset and retrieve information about the uploaded CSV file.

The activity successfully extracted metadata such as:

- File Name
- File Size
- Last Modified Timestamp
- File Existence Status
- Other File Properties

This validation step ensured that the file was available and accessible before performing data movement operations.

Metadata validation is a critical step in real-world ETL processes because it helps prevent failures caused by missing or incorrect source files.

Building the Data Movement Pipeline

After successful metadata validation, a data ingestion pipeline was developed using the Copy Data activity.

The pipeline workflow consisted of:

Source Configuration

- Source Type: Azure Blob Storage
- Dataset: sales_data.csv

Destination Configuration

- Destination Dataset: sales_data_destination

Data Flow Process

1. Azure Data Factory reads the CSV file from Blob Storage.
2. Data is processed by the Copy Data activity.
3. Data is transferred to the configured destination dataset.
4. Pipeline logs and execution metrics are generated automatically.

The Copy Data activity is one of the most commonly used ADF activities because it enables movement of data across different storage systems and databases.

Access Control and IAM Configuration

To ensure secure access between Azure services, Identity and Access Management (IAM) configurations were reviewed.

The following Azure role assignments were considered:

Reader Role

Provides read-only access to Azure resources.

Contributor Role

Provides permission to create, modify, and manage resources without granting ownership privileges.

Appropriate permissions were assigned to ensure that Azure Data Factory could access and interact with Azure Storage resources successfully.

Implementing IAM roles helped maintain security while allowing required operations to be performed.

Pipeline Execution

Once all configurations were completed, the pipeline was executed.

The execution process involved:

1. Validating the pipeline.
2. Publishing all changes.
3. Running the pipeline using the Debug option.
4. Triggering the pipeline manually.
5. Monitoring execution results.

During execution, Azure Data Factory successfully:

- Connected to Blob Storage.
- Retrieved file metadata.
- Read the CSV file.

- Processed the data.
- Completed data transfer.

No errors were encountered during execution.

Monitoring and Results

The Monitor section of Azure Data Factory was used to track pipeline execution.

The monitoring dashboard displayed:

- Pipeline Name
- Start Time
- End Time
- Duration
- Trigger Type
- Execution Status

Multiple executions of the MetadataPipeline were performed, and each run completed successfully.

Observed Results

Parameter	Result
Pipeline Name	MetadataPipeline
Execution Status	Succeeded
Trigger Type	Manual Trigger
Average Duration	24–25 Seconds
Metadata Retrieval	Successful
Data Access Validation	Successful
Pipeline Monitoring	Successful

The successful execution confirmed that all Azure services were integrated correctly and functioning as expected.

Learning Outcomes

Through this project, I gained practical knowledge of:

- Azure Resource Group management
- Azure Storage Account creation
- Azure Blob Storage operations
- Azure Data Factory architecture
- Linked Services configuration
- Dataset creation and management
- Metadata extraction using Get Metadata activity
- Data movement using Copy Data activity
- Azure IAM and access control
- Pipeline execution and monitoring
- Cloud-based ETL workflow implementation

This project provided valuable hands-on exposure to data engineering concepts and Azure cloud services used in modern enterprise environments.

Conclusion

The project successfully achieved its objective of creating an end-to-end data pipeline using Azure Blob Storage and Azure Data Factory. The workflow included resource provisioning, data storage, metadata validation, pipeline development, data movement, access management, and execution monitoring.

The uploaded CSV file was successfully stored in Azure Blob Storage, validated using the Get Metadata activity, and processed through Azure Data Factory pipelines. All pipeline runs completed successfully with a status of Succeeded, demonstrating the reliability and effectiveness of Azure's cloud-based data integration services.

Overall, this project provided a comprehensive understanding of how Azure services work together to build scalable, secure, and efficient data pipelines for real-world data engineering applications.